

HearthGen: Knowledge-Graph-Augmented Retrieval for Hearthstone Card Art Generation

COMP 646 Project Team
Rice University

GitHub: <https://github.com/TraceOnSnow/HearthstoneCardGenerator>

Abstract

*Generating convincing Hearthstone-style card art requires more than visual style transfer: the artwork should also reflect card mechanics, class identity, and recurring card-family conventions. We present **HearthGen**, an end-to-end prototype that converts Hearthstone card data into structured semantics, builds a semantic knowledge graph (KG), retrieves mechanically grounded reference cards, and uses these references to guide Stable Diffusion and a Hearthstone LoRA adapter. Our system supports both artwork generation and KG-augmented card design from natural-language user requests. We evaluate retrieval against TF-IDF and CLIP baselines, and evaluate generation with qualitative side-by-side examples and automatic proxy metrics. The main result is that KG retrieval provides more interpretable design evidence than visual or lexical similarity alone, especially for implicit mechanics and meme/card-family requests such as a defensive “Rager” card.*

1. Introduction

Trading-card artwork generation is a constrained creative task. A generated Hearthstone card should match the game art style, but it should also express the card’s class, mechanics, and design lineage. For example, a user request such as “a defensive Control Warrior Rager meme card” is not just an image prompt. It refers to the Magma Rager family, the traditional 3-mana 5/1 statline, and Warrior armor mechanics. A generic text-to-image model or a simple CLIP nearest-neighbor search may retrieve visually plausible images, but it has no explicit representation of these gameplay relations.

We build **HearthGen**, a retrieval-augmented generation pipeline for Hearthstone card art. The system starts from Hearthstone JSON data and converts each card into a structured semantic record containing identity fields, stats, actions, keywords, generated-card references, derived-card

links, and a LoRA caption. From these records we construct a semantic KG. User requests are parsed into structured queries and matched against the KG to retrieve reference cards. The retrieved evidence is then used to guide card design and image generation with Stable Diffusion [8] and a LoRA adapter [3].

Our contributions are:

- A structured semantic representation for Hearthstone cards that combines deterministic metadata, rule-based action extraction, and optional LLM enrichment.
- A semantic KG retrieval pipeline that returns faceted evidence rather than a single flat ranked list.
- A generation and evaluation workflow comparing SD text-only, LoRA text-only, and reference-conditioned generation using TF-IDF, CLIP [7], and KG retrieval.

2. Related Work

Our work combines retrieval-augmented generation, knowledge graphs, and diffusion-based image synthesis. Retrieval-augmented generation (RAG) uses external evidence to condition generation rather than relying only on model parameters [4]. In our setting, the retrieval corpus is not text passages but structured card semantics and artwork references. Knowledge graphs provide an interpretable way to represent entity and relation structure; we use the graph to connect cards to classes, keywords, actions, resources, and generated-card references.

For image generation, we use latent diffusion models [8], which generate images in a compressed latent space, and LoRA [3], a parameter-efficient finetuning method. CLIP [7] is used both as a retrieval baseline and for automatic prompt-alignment metrics. We also use DINOv2 [6] or CLIP image embeddings as proxy style/reference similarity metrics. The card data is derived from Blizzard’s Hearthstone API and community-maintained data sources, including HearthstoneJSON and HearthSim tooling [1, 9, 2].

3. Method

3.1. Structured Card Semantics

The input is Hearthstone card JSON. We construct one semantic record per card, including collectible and non-collectible derived cards. Deterministic fields are extracted directly from JSON and metadata maps: card type, class, set, rarity, spell school, minion type, mana cost, attack, health, keywords, and child card IDs. Rule-based text extractors identify common gameplay actions such as `deal_damage`, `summon`, `draw`, `gain_armor`, `freeze`, and `destroy`. Each action stores type, amount, target, resource, trigger, condition, and raw text phrase when available.

We optionally enrich this base representation using MiniMax-M2.7 [5]. The LLM is not the source of deterministic facts; it only fills fields that rules often miss, such as generated-card roles, implicit conditions, action groups, and concise semantic summaries. The same semantic record is used for two downstream artifacts: a LoRA caption for generation and graph triples for KG retrieval.

3.2. Semantic Knowledge Graph

The KG contains card nodes and value nodes. Important node types include class, card type, keyword, action, target, resource, mechanic tag, generated card name, and related card name. Edges include `HAS_CLASS`, `HAS_KEYWORD`, `PERFORMS_ACTION`, `AFFECTS_RESOURCE`, `GENERATES_CARD`, and `HAS_CHILD_CARD`. Visual tags are intentionally excluded from the KG facts because they are derived from text and should not be double-counted as gameplay evidence.

Natural-language requests are parsed into structured queries. A query contains fields such as class, card type, actions, resources, keywords, generated card names, and related card names. Retrieval first scores cards by matching query fields to KG nodes. However, a single flat ranking can over-rank repeated mechanic matches. Therefore, for card design we return a faceted evidence package: family/name anchors, mechanic matches, identity matches, and a diversified shortlist. This is important for requests such as a Rager meme card, where a lower-ranked Magma Rager anchor may be more useful than another high-scoring Warrior armor card.

3.3. Generation Pipeline

For image generation we compare Stable Diffusion v1.5 text-only generation, LoRA text-only generation, and reference-conditioned generation. The LoRA adapter is trained on Hearthstone artwork captions. Reference conditioning is implemented with image-to-image generation. We found that using multiple strong references can create

Method	Align.↑	Style↑	Ref.↑
SD text-only	0.296	0.750	–
SD + KG ref	0.295	0.837	0.796
LoRA text-only	0.301	0.858	–
LoRA + KG ref	0.297	0.861	0.830

Table 1. Generation proxy metrics on 19 DIY prompts. Alignment is CLIP prompt-image similarity; Style is similarity to Hearthstone art references; Ref. is similarity to the selected KG reference image.

Retrieval	Rel.@5↑	Main failure mode
TF-IDF	0.679	literal keywords
CLIP	0.456	visual but not mechanic match
KG	0.806	repeated facet over-ranking

Table 2. Retrieval proxy evaluation on 19 DIY prompts. KG returns interpretable reasons and faceted evidence, which is useful even when a flat ranking is imperfect.

“fusion” artifacts; the preferred policy is one primary visual reference plus text-only semantic hints from the rest of the KG evidence.

4. Experiments

We evaluate two questions. First, does KG retrieval return more useful Hearthstone references than lexical or visual baselines? Second, do LoRA and KG references improve card artwork generation?

4.1. Retrieval Evaluation

We compare three retrieval methods: TF-IDF over captions, CLIP text-to-image nearest-neighbor retrieval, and semantic KG retrieval. The queries include explicit mechanics, such as a Warlock lifesteal damage spell, and harder implicit requests, such as cards that generate Solar Eclipse and Lunar Eclipse or a defensive Rager meme card. Retrieval outputs top- k reference cards with explanations.

4.2. Generation Evaluation

For generation, we compare SD text-only, SD with references, LoRA text-only, and LoRA with KG references on the same prompts. We report automatic proxy metrics: CLIP prompt alignment, style similarity to Hearthstone references, and reference similarity. These metrics do not fully measure card-design correctness, so we also include qualitative side-by-side examples and failure analysis.

5. Results and Analysis

Figure 1 should show the complete system in one glance. The key point is that the KG is not only a way to find images. It also returns design evidence. In the Rager example,

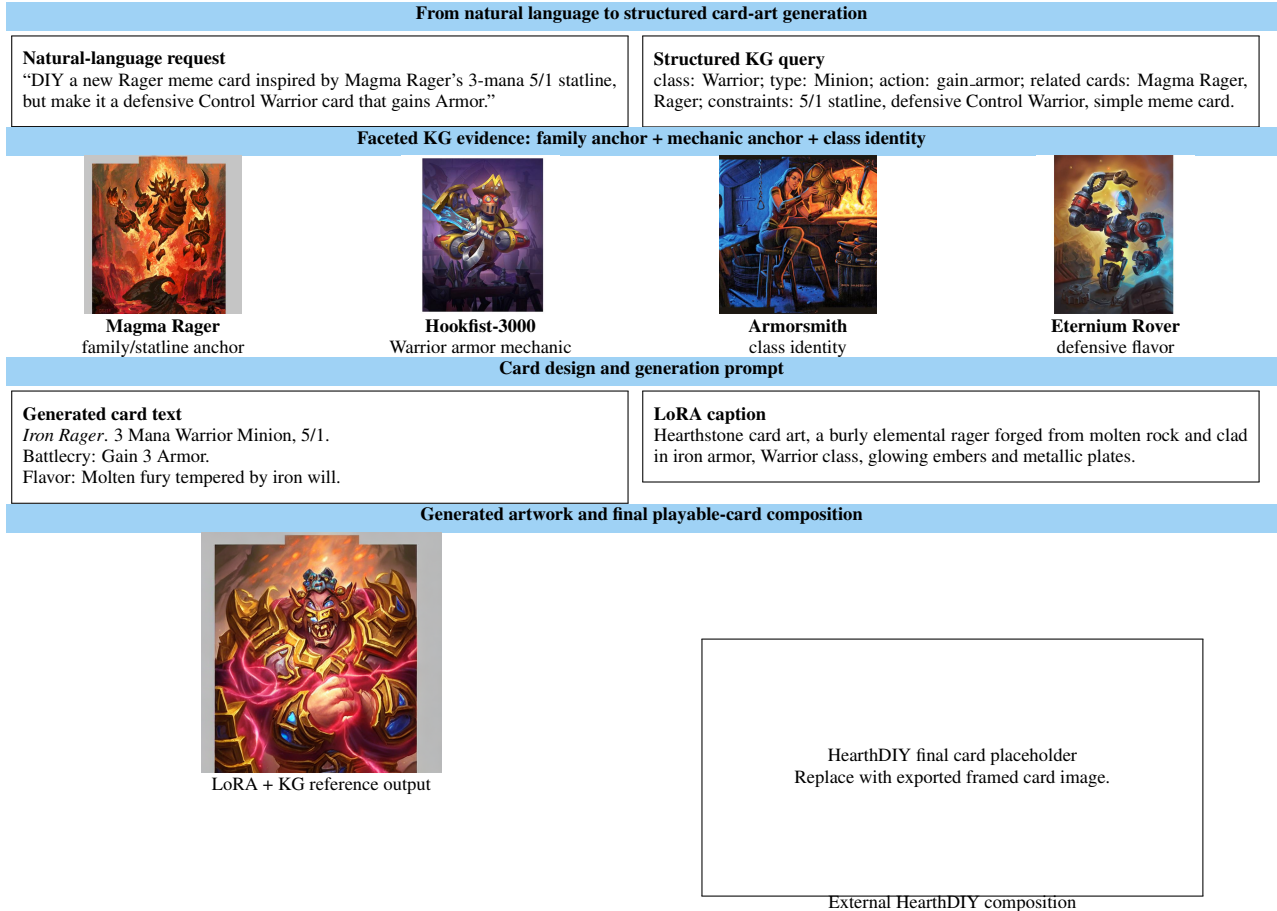


Figure 1. **End-to-end HearthGen case study.** A vague community-style request is converted into structured semantics, matched against faceted KG evidence, turned into a playable card design and LoRA caption, and finally rendered as Hearthstone-style artwork.

the user request is parsed into a Warrior minion with armor gain and related cards Magma Rager/Rager. The KG retrieves both Warrior armor cards and Rager-family anchors. The card-design module then produces *Iron Rager*, a 3-mana 5/1 Warrior Elemental with “Battlecry: Gain 3 Armor.” This illustrates why faceted evidence is preferable to a single score-sorted list.

Figure 2 should show the generation ablation. The expected pattern is that SD text-only images often follow the prompt but lack Hearthstone style; LoRA text-only improves style but may miss specific mechanics; reference-conditioned outputs better match the selected card theme. KG references are most useful when the prompt depends on gameplay semantics or generated-card relations rather than surface-level visual similarity.

Tables 1 and 2 provide quantitative evidence. The automatic generation metrics are proxy measures: CLIP alignment checks prompt-image compatibility, while style and reference similarity check image embedding proximity. These numbers should be interpreted together with the qualitative grids, because a high reference similarity can

also indicate over-copying. We observed that overly strong image-to-image conditioning can create fusion artifacts, so our final design recommends using one primary reference and treating other KG hits as text-only semantic hints.

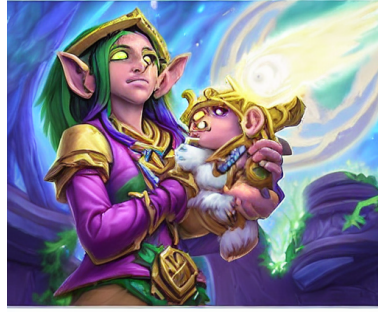
6. Conclusion

HearthGen is an end-to-end prototype for KG-augmented Hearthstone card art generation. The main contribution is not a new diffusion architecture, but a structured semantic KG that retrieves mechanically grounded and interpretable references for generation and card design. LoRA improves visual style, while KG retrieval improves the relevance of prompts and references. The current system is limited by small-scale evaluation and imperfect reference conditioning, but it demonstrates a practical path from raw card data and user requests to structured card semantics and generated Hearthstone-style artwork.

Acknowledgments. We acknowledge the use of AI coding assistants for implementation support, debugging, and drafting assistance. All experiments, analysis, and final system design decisions were reviewed by the project team.



Iron Rager
3 Mana Warrior Minion, 5/1. Battlecry: Gain 3 Armor.
Vague meme/family request



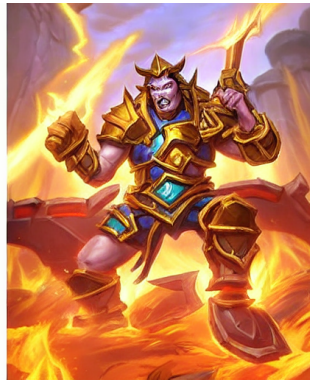
Eclipse Weaver
4 Mana Druid Minion, 3/5. Battlecry: Add Solar Eclipse and Lunar Eclipse to your hand.
Implicit related-card request



Silver Hand Commander
5 Mana Paladin Minion, 4/5. Battlecry: Summon two 1/1 Silver Hand Recruits. Your Silver Hand Recruits have +1 Attack.
Token/archetype request



Frozen Tempest
5 Mana Mage Spell. Deal 2 damage to all enemy minions. Freeze them until end of turn.
Mechanic-first request



Iron Oathbreaker
3 Mana Warrior Weapon, 3/2. After your hero gains Armor, this weapon has +1 Attack until end of turn.
Fully specified card



Chrono-Bronze Drake
6 Mana Neutral Dragon, 5/7. Battlecry: Discover a card. If you control a Mech, reduce its Cost by (2).
Art-first request

Figure 2. **LoRA + KG-reference qualitative outputs.** Six representative user prompts are shown as final generated artwork with the corresponding card description. The set intentionally mixes vague creative requests, implicit Hearthstone references, mechanic-first briefs, fully specified card text, and art-first requests.

References

- [1] Blizzard Entertainment. Hearthstone game data api. <https://develop.battle.net/documentation/hearthstone/game-data-apis>, 2026. Accessed May 2026.
- [2] HearthSim. Hearthsim python hearthstone data. <https://github.com/HearthSim/python-hearthstone-data>, 2026. Accessed May 2026.
- [3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [5] MiniMax. Minimax m2.7 text generation api. <https://platform.minimaxi.com/docs>, 2026. Accessed May 2026.
- [6] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [7] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.
- [8] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.
- [9] Zero-to-Heroes. Hearthstonejson. <https://github.com/Zero-to-Heroes/HearthstoneJSON>, 2026. Accessed May 2026.